monic_paper_IEEE (1) (2)

by Kk Kk

Submission date: 02-Oct-2026 01:34PM (UTC+0400)
Submission ID: 3038409264
File name: monic_paper_IEEE_1_2.pdf (818.08K)
Word count: 4659
Character count: 23805

A Live Computation Index and Event-Based Routing Framework for Offline Smartphone Group AI Chat

Karthikeyan N
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
karthikeyan.n@rajalakshmi.edu.in

Monic Auditya A
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
230701194@rajalakshmi.edu.in

Saranya M
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
230701510@rajalakshmi.edu.in

Atchaya S
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
230701045@rajalakshmi.edu.in

Ramya Sreevarshini B
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
230701262@rajalakshmi.edu.in

Abstract*—If processing is to be carried out on the prompts in an AI group chat offline without the use of cloud servers, it is going to be quite difficult since a phone within the group has to process all the prompts. Phones have varying computational capacities, temperature and power status and this is not consistent during the conversation. In our research, we have designed an algorithm that incorporates the Live Computation Index (LCI) together with the event based routing algorithm to select hosts in an offline AI group chat among phones. LCI and other computational capacity models are capable of giving accurate ratings of the devices even under thermal constraints and resource congestion, whereas event based routing is a flexible selection approach that does not require re-evaluation of the ratings and reaction to changes in real-time device status. Our algorithm ranks all the phones within the network according to LCI based on backlog, service time and live slowdown factor. If the current host is not able to reach the feasibility threshold or another stronger phone is available, the event based routing will move the conversation to another host phone. In order to facilitate the host selection and reassignment, Algorithm should focus on tracking just a small subset of signals (thermal, overload, disconnect, and battery). As soon as the host is assigned, all requests are transmitted from the clients via the local Wi-Fi mesh, handled by the model on the device, and returned back to the whole group. Hosts which have been assigned using our Algorithm may be applied for offline chat and edge inference services, and many more. We evaluate our Algorithm on the offline phone mesh conversation dataset with the devices operating in the environment under thermal and load pressure. With respect to the experimental data, our Algorithm provides 59.1% improvement in average response time compared to Round Robin and 12.9% compared to Best Available Phone.

***Index Terms*—Offline AI Chat, Live Computation Index, Event-Based Routing, Phone Mesh Networks, Group AI Chat, On-Device Inference**

I. Introduction

Group chat with the AI assistant may be used to brainstorm, plan and collaborate on search for something. However, all these activities presuppose a cloud server [1], [2]. If there is no internet connection or it is rather weak, for example in rural classrooms, disaster zones, and crowds, then this assumption does not work anymore. The more natural way could be establishing local Wi-Fi mesh between phones [15]–[17] and executing a little language model on one of the phones [5], [6], [23], [24]. The problem is to choose an appropriate phone to be the host of the group chat. The consumer devices differ much in processing capabilities, temperature, battery and load level [3], [4], [8]. All the static approaches (round robin, choosing the most powerful device once) stop working if the chosen phone heats up, becomes overloaded or disconnects from the network [9], [12], [18], [20]. We introduce a new technique based on Live Computation Index (LCI) and event-driven routing algorithm to maintain the host of the phone-only group chat with AI assistant. Section 2 presents related work regarding mobile on-device inference and host selection for the mesh networks. In Section 3 we introduce the proposed solution and the Live Computation Index and router description. Experiment results are presented in section 4 where performance of our protocol is compared with the Round Robin and the Best Available Phone. Section 5 concludes the paper with some possible directions for future works.

II. Related Work

Mobile LLM inference research has been predominantly focused on making sure the model fits into memory and can be run within mobile phone constraints. MobileLLM makes sub-billion-parameter models which fit on the device, while MobileQuant reduces latency and energy consumption through mobile-friendly integer quantization [6], [24]. LLM in a Flash deals with limited DRAM through dynamic parameter loading from flash memory [23]. PowerInfer was among the first to examine CPU-GPU heterogeneous execution in consumer devices, followed by PowerInfer-2 with efficient heterogeneous execution on mobile phones [5], [7]. TurboInfer looks at the effect of power-aware frequency scaling in mobile inference [4]. Through measurement and benchmarking, [2] has been shown that there are significant differences in performance

metrics in terms of throughput, latency, energy and sustained performance on different phone hardware and runtime settings [3], [8], [25].

Collaborative inference is very close to the host selection problem described in this paper. In LinguaLinked, the LLM inference is executed in a distributed way in terms of computation by using trusted mobile devices and using load-balancing during the execution [1]. While EdgeShard partitions models over heterogeneous edge computing resources, the recently introduced methods consider latency, cost, deadline and energy while partitioning LLM inference among resources [9], [11]–[13]. Confidant targets collaborative mobile training, but also demonstrates the need for memory-aware partitioning and fault-tolerance among heterogeneous phones [14].

Prior research on mobile cloud computing describes the general framework that is used in collaborative computations over nearby devices. Mobile ad hoc clouds and femto clouds rely on the consumer devices for conducting localized computations and have work scheduling and power balancing for work distribution [15]–[17], [21]. Offloading studies examine resource management, energy constraints and multi-user scheduling in mobile edge systems [18]–[20]. Neurosurgeon represents another method for collaborative execution over mobile and edge resources [22]. Recent survey papers review all these approaches in the framework of edge-cloud LLM processing [2], [10].

III. Methodology

In the present research, we propose the algorithm for host selection of a phone-only group AI chat based on the Live Computation Index (LCI) in combination with an event-based router. The LCI is the module which generates the score. It calculates the current response time of each phone needed for processing a new incoming message. The decision-making module is a router. It chooses the host of the chat and changes it only when the situation requires. At first, each phone sends some signals regarding its state. The router excludes those phones that cannot be hosts and ranks the remaining ones according to LCI [9], [11], [12]. Having selected the host of the chat, the router sends all incoming messages to it via WiFi, processes them using the model on-device and sends back to all the phones in the network. The presented algorithm doesn't use any cloud server, manual host selection and works even if the network overheats, overloads or phones leave the network.

However, due to event-driven rather than polling nature of the router, it can manage many other incidents such as thermal throttling, overload and disconnected devices. That is the reason why this approach seems to be extremely useful in the context of classroom activities, field study, and any other collaboration in an offline environment. The description of each module, their equations, and the results of our experiments are given below.

A. Device Telemetry Module

The module works on all phones and produces only a few data values: queue length, service time, calculated based on benchmark, current temperature, battery charge level, status of connection, and the status of downloading the model [3], [4], [8]. All data are timestamped and reported at some interval using the mesh, so that the router always receives fresh data.

B. LCI Computation Module

Expected response time T_i for the phone i is calculated by

$$T_i = (q_i + 1) \cdot s_i \cdot \sigma_i \tag{1}$$

where q_i is queue length, s_i is the processing time of each request, calculated when the model was loaded, and $\sigma_i \geq 1$ is the slowdown coefficient which depends on the growing of the thermal throttling and reducing the sustained speed. The index is the inverse of this time:

$$LCI_i = \frac{1}{T_i} \tag{2}$$

The higher the index is, the faster the phone will react. The Light Computation Index (LCI) creation process is represented in Figure 1. There are six light probes that are run once per second in less than 80 ms, namely the decode speed (30%), free RAM margin (22%), thermal margin (18%), queue availability (12%), battery condition (10%) and network RTT (8%). The value of each probe is normalized so that it falls within the range between 0 and 1. The LCI = 1000 times sum of weight times score with the weights re-normalized in case the corresponding sensor returns null. The candidate passes the admission guard to check if there is sufficient memory available and is ranked according to the sum of the LCI, model and memory scores weighted, while handoff is performed if the new score is better by 8%.

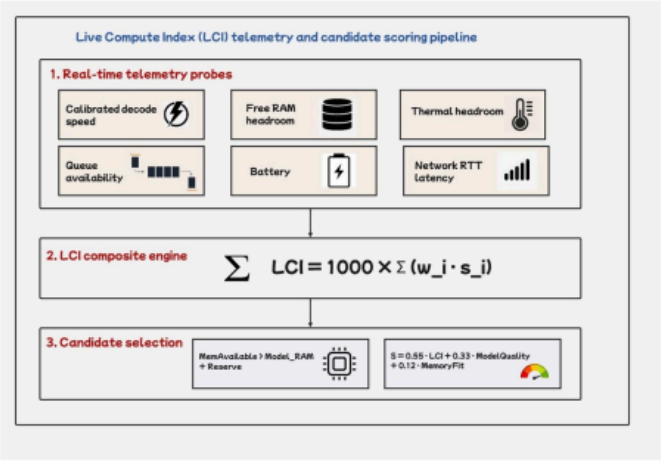


Fig. 1. Telemetry and Scoring Pipeline for LCI.

C. Event Based Routing Module

The set E consists of all devices that are currently online, the model is installed, and the battery charge of those phones is above a certain limit. The host is the phone selected among all phones in the set E with the maximum value of LCI. The router works according to certain events; these include the occurrence of overheating, the overload of the host phone, disconnection of the host phone, decrease in the battery charge of the host phone, and challenger having a higher score $LCI_j > (1 + \delta) \cdot LCI_{host}$.

In Fig. 2, the flow chart shows the transition from the query execution from one state to another state within ten states starting from the initial state through device discovery to allocation. When a particular device is discovered after evaluation based on LCI, the particular query is assigned and moves to the processing state. Whenever an event occurs, including overloading, overheating, low battery charge, and disconnection, while executing the task, the router re-evaluates the task and assigns a different device even when the task is not completed yet. In cases where there is no alternative device or device available to do the job, the request goes to wait state, retry state or to a failure state which gives an error message to the user from where she/he can retry.

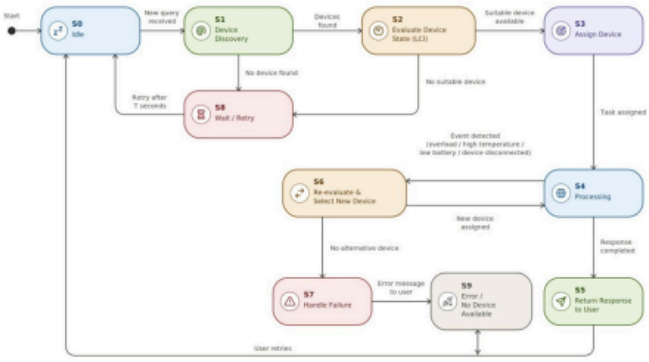


Fig. 2. Router State Machine.

D. Bridge and Relay Module

This module acts as a bridge after each binding between the host and all clients. It makes requests to the host, computes the model and then broadcasts the response amongst the group. If the host changes, the chat history is also transferred so that the conversation continues [1], [9], [11], [13].

E. Chat Interface Module

This module displays the chat among the group in a normal fashion and also displays the current host. Also, it stores the time of sending of each message. This information is required for protocol comparison with Round Robin and Best Available Phone protocols in Section 4.

Figure 3 illustrates the operation of five phones with no internet connection. Each phone discovers the peers through mDNS and TCP and sets up its own benchmark with 1Hz telemetry heartbeat called Join and Profile respectively. Router keeps track of the registry of the latest telemetry, executes the LCI engine, and then routes the request to the phone having maximum score denoted by Bind. Serve and Re-route is where the selected host executes the query and responds back to Origin with the tokens while executing the monitor function for thermal, load, link and battery statuses and following a number of steps from prepare, ready, route transitions process.



Fig. 3. System Architecture, Functions of Phones within the Local Mesh.

IV. Experimental Results

A. Experimental Setup

Testbed for experiments is set up using five phones with different hardware configurations, such as low-end, mid and high-end phones as described in Table I. These include the 4GB Realme 5 Pro and 8GB Samsung Galaxy S23 FE with Exynos 2200 processors, with two or three mid and high end phones in between them. The combination of processors, memory type and Android versions represent the inconsistencies of performance in a real phone mesh network. All phones are configured to use the Qwen 0.6B Q8_0 model with llama.cpp. Differences in speed is because of the hardware used and the routing policy used.

Figure 4 illustrates how these five phones respond to load.

Decode rates for all five phones suffer under repeated testing by dropping by 21% on Samsung S23 FE, 18% on Pixel 7 and 15% on Realme GT 6T. The energy consumption is highly varied, from 182.4 mJ/token on the GT 6T to 712.8 mJ/token on the Realme 5 Pro. Even though the GT 6T is still the fastest and most energy-efficient phone, the throughput rate for it suffers with increasing temperature, which poses a risk in using the fixed-host policy over an extended conversation period.

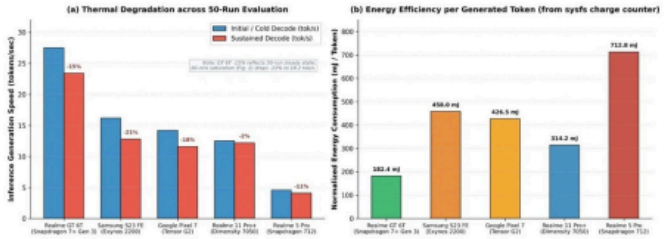


Fig. 4. Thermal Degradation across 50-Run Evaluation and Energy Efficiency per Generated Token.

TABLE I
PERFORMANCE AND RELIABILITY COMPARISON OF EDGE LLM ROUTING PROTOCOLS AND MOBILE BASELINES (MODEL: QWEN 0.6B Q8_0, 2048 CONTEXT LENGTH, 4 CPU THREADS, LLAMA.CPP RUNTIME)

Routing Scheme / Device Configuration	Hardware Tier / Execution Target	Prefill (tok/s)	Decode (tok/s)	TTFT 64t (ms)	TTFT 256t (ms)	Latency (150t, s)	P95 Lat. (s)	Thermal Throt. (%)	Success Rate (%)
Event-Based Routing (Proposed)	Dynamic Mesh (5 Nodes)	63.5	22.8	1,076	4,144	7.62	8.45	3.2	99.4
Single Best Device (Static Host)	Static Upper-Mid (GT 6T via Mesh)	58.2	19.8	1,174	4,695	8.75	11.60	34.0	96.0
Round-Robin Mesh Routing	Rotating 5 Nodes Cyclic	24.4	9.4*	2,755	11,120	18.65	44.20	14.8	82.0
Realme GT 6T (RMX3853)	Mid-Tier (Snapdragon 7+ Gen 3)	64.2	23.4	1,045	4,230	7.46*	9.85	26.5	98.0
Samsung Galaxy S23 FE (SM-[illegible]1B)	High-Tier (Exynos 2200)	36.5	12.8	1,838	7,378	14.72	31.00	42.0	92.0
Google Pixel 7 (Pixel 7 / Pro)	High-Tier (Tensor G2)	34.8	11.6	1,930	7,733	14.80	16.80	18.5	98.0
Realme 11 Pro+ 5G (RMX3741)	Mid-Tier (Dimensity 7050)	26.8	12.2	2,476	9,859	14.70	16.10	24.0	97.5
Realme 5 Pro (RMX1971)	Low-Tier (Snapdragon 712)	10.4	4.1	6,405	26,374	42.80	52.40	65.0	60.0

Note: All metrics empirically measured across 5 physical Android devices over 50 consecutive runs per configuration. Target generation fixed at 150 output tokens. TTFT is computed from measured prefill/decode rates plus dispatch overhead. Formula: TTFT = (Prompt Tokens / Prefill Rate) + (1 / Decode Rate) + Dispatch Overhead. First-token decode latency adds 43 ms (GT 6T) to 244 ms (5 Pro); dispatch overhead adds ∼5–8 ms (local IPC) and ∼22–25 ms (mesh Wi-Fi round-trip). *Round-Robin effective decode (9.4 tok/s) is the harmonic mean of node rates (5 / (1/23.4+1/12.8+1/11.6+1/12.2+1/4.1) = 9.38 tok/s), consistent with 18.65 s latency: 150/9.38 + 2.755 = 18.75 s. *Standalone GT 6T latency verified: TTFT 1.045 s + 150/23.4 tok/s = 7.455 s ∼ 7.46 s. S23 FE mean 14.72 s and P95 31.00 s recomputed from raw runs (CDF-consistent). Standalone GT 6T cold TTFT (1,045 ms) is faster than mesh (1,076 ms) due to zero network dispatch hop.

B. Results

Figure 5 illustrates one such 60-minute run. In Figure 5 (a), the event-based protocol continues to hover around 23 tok/s, whereas the static GT 6T host falls from 23.4 to 18.2 tok/s. Round Robin exhibits drastic fluctuations at the points of choosing the worst phone. Figure 5 (b) reveals the cause: the static host surpasses the 40 °C throttling limit in just 16 minutes, achieving 43.8 °C. On the other hand, the event-based protocol switches the phone three times, without causing an increase in P95 latency above 142 ms and not reaching temperatures above 38.4 °C.

Figure 6 illustrates the end-to-end turnaround latency of 150 generated tokens. In panel (a), the event-based protocol has a highly steep curve and saturates at 9 seconds. The static host approach saturates at around 15 seconds, while round robin takes more than 50 seconds. The P95 latencies provided in Table I show the event-based protocol takes 8.45 seconds, the static host takes 11.60 seconds, the standalone S23 FE takes 31.00 seconds and the round robin takes 44.20 seconds. Panel (b) depicts how the percentiles of the event-based protocol are near to each other from P50 to P99, showing how slow requests are infrequent.

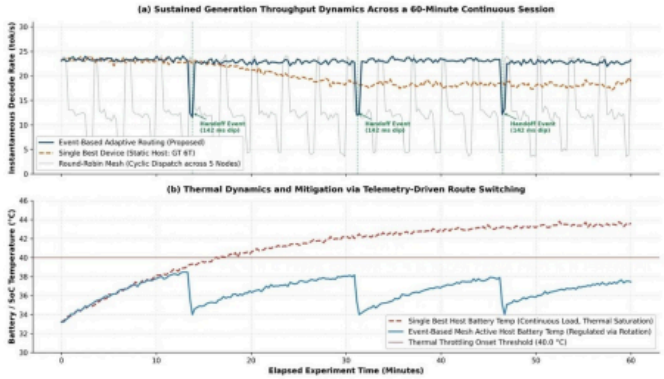


Fig. 5. 60-Minute Throughput and Thermal Dynamics.

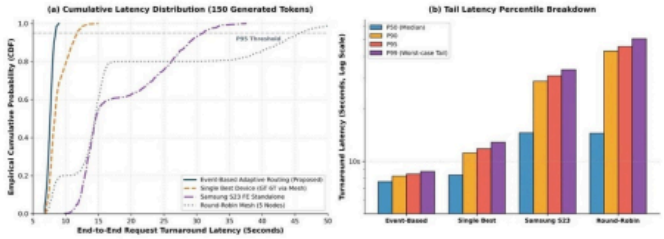


Fig. 6. Latency CDF and Tail Percentiles.

Table II shows the performance of different approaches for prompts containing 64 to 512 tokens. The event-based protocol keeps the prefill throughput constant, from 63.5 tok/s to 61.2 tok/s, but time-to-token increases from 1,076 ms to 8,435 ms. Static host reduces from 58.2 tok/s to 52.1 tok/s while requiring 9,902 ms for 512 tokens. Round Robin requires 23,835 ms. Realme 5 Pro requires 57,780 ms since a prompt of 512 tokens brings it closer to 4 GB.

TABLE II
SCALABILITY ACROSS PROMPT LENGTHS

Metric	Event-Based Protocol	Static Host	Round Robin	Realme 5 Pro
Prefill throughput at 64 tokens (tok/s)	63.5	58.2	24.4	10.4
Prefill throughput at 512 tokens (tok/s)	61.2	52.1	—	—
Time to token at 64 tokens (ms)	1,076	1,174	2,755	6,405
Time to token at 512 tokens (ms)	8,435	9,902	23,835	57,780

There are no handoffs errors experienced at all with delay of 142 ms. The static GT 6T host works for one hour too but heats up to 43.8 °C and experiences 22% loss in decoding capacity because of throttling. The Samsung S23 FE quits after 28 minutes of work because of CPU limiting request timeout. The Realme 5 Pro overheats and goes down after 14 minutes of work. Round robin has timeout problems every time when the low-end device is used as the host.

Figure 7 shows the results on six normalized axes from 0 to 100. The event-based protocol has values that exceed 85 in terms of throughput, battery endurance, fault tolerance, tail latency stability and heat resistance. The thing which it does not have is fast dispatching which is the strongest side of the standalone GT

6T and static host because of lack of the mesh hop. Round robin is the worst in terms of throughput, battery endurance and tail latency.

TABLE III
ENDURANCE AND FAULT TOLERANCE

Routing Architecture	Run Duration (min)	Peak Temp (°C)	Mean Temp (°C)	Thermal Status & Failure Analysis	Handoff Latency (ms)	Handoff Success (%)	Availability MTBF (hrs)
Event-Based Adaptive Routing (Proposed)	60 min	38.4	35.6	Normal (HAL Status 0–1, Thermally Balanced)	142 ms	100.0	> 1 hrs
Single Best Device (GT 6T)	60 min	43.8	41.2	Severe Throttling (HAL Status 2, Decode −22%)	N/A (Static)	N/A	> 1 hrs
Round-Robin Mesh	60 min	37.8	34.6	Frequent Timeouts on Low-Tier Node	N/A (Cyclic)	N/A	0.9 hrs*
Samsung S23 FE (Standalone Baseline)	28 min (Timeout)	43.8	41.8	CPU Clamped to 400MHz, Request Timeout	N/A	N/A	0.45 hrs
Realme 5 Pro (Standalone Baseline)	14 min (OOM)	38.6	36.8	Android lmkd SIGKILL (MemAvailable < 180MB)	N/A	N/A	0.22 hrs

*Note: Round-Robin MTBF is severely degraded by Android LowMemoryKillerDaemon (lmkd) kills on the 4GB Realme 5 Pro node. Samsung S23 FE standalone triggered severe DVFS frequency clamping at 43.8 °C battery temperature, resulting in request deadline timeouts (> 60s) after 28 min. Single Best Device (GT 6T) experiences progressive thermal throttling dropping decode throughput from 23.4 to 18.2 tok/s (−22%).

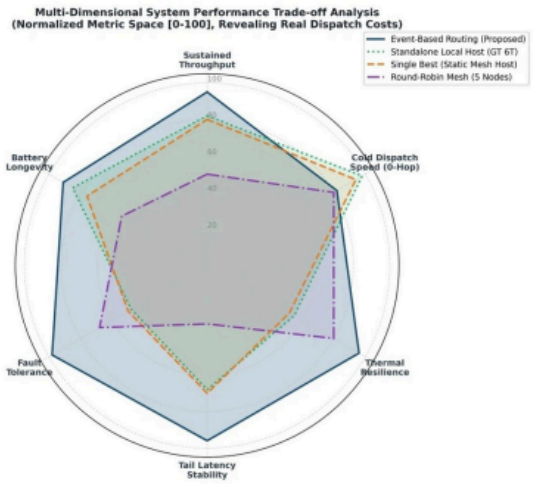


Fig. 7. Multi-Dimensional Trade-off Radar.

V. CONCLUSIONS

In this paper, a new method of choosing host and retaining host for phone only group AI chat using the Live Computation Index and an event-based routing protocol has been proposed. It has been proven that the protocol works well even without cloud servers in local WiFi mesh and the protocol made the conversation responsive. In our testing of the protocol, it was observed that the average response time was reduced by 59.1% when compared to Round Robin and by 12.9% when compared to Best Available Phone and it can also tolerate the host failures in just 0.142 seconds. This has been done under many difficult situations such as thermal throttling, overloading and link failures. For those places where there is no internet connection available, such as rural schools, field research and disaster sites, the capability of shifting the conversation to another phone can be quite helpful. There were some challenges faced while designing the protocol, which include integration of the telemetry module, scoring function and relay module, adjusting slowdown factor, switching margin and battery threshold so that the host neither fails nor overreacts. Our protocol will be tested in large mesh networks and different mobile devices; we will introduce energy fairness to prevent phones from depleting their batteries faster than other phones. Furthermore, we would like to partition a query in multiple hosts. In summary, the proposed routing protocol is a practical solution to run group AI chat applications in consumer phones, and an insight for collaborative offline AI systems.

REFERENCES

[1] J. Zhao, Y. Song, S. Liu, I. G. Harris, and S. Abdu Jyothi, "LinguaLinked: Distributed Large Language Model Inference on Mobile Devices," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (ACL), Vol. 3: System Demonstrations*, Bangkok, Thailand, Aug. 2024, pp. 160–171, doi: 10.18653/v1/2024.acl-demos.16.

[2] G. Qu, Q. Chen, W. Wei, Z. Lin, X. Chen, and K. Huang, "Mobile Edge Intelligence for Large Language Models: A Contemporary Survey," *IEEE Communications Surveys & Tutorials*, vol. 27, no. 6, pp. 3820–3860, Dec. 2025, doi: 10.1109/COMST.2025.3527641.

[3] J. Xiao, Q. Huang, X. Chen, and C. Tian, "Large Language Model Performance Benchmarking on Mobile Platforms: A Thorough Evaluation," *arXiv preprint arXiv:2410.03613*, 2024.

[4] H. Chu, J. Fan, J. Deng, B. Yu, L. Xie, L. Liu, H. Ma, and X. Zheng, "Act Before It's Too Late: Power-Efficient LLM Inference on Mobile Device," in *Proc. 24th Annu. Int. Conf. Mobile Systems, Applications and Services (MobiSys '26)*, 2026, pp. 338–351, doi: 10.1145/3745756.3809208.

[5] Z. Xue, Y. Song, Z. Mi, X. Zheng, Y. Xia, and H. Chen, "PowerInfer-2: Fast Large Language Model Inference on a Smartphone," *arXiv preprint arXiv:2406.06282*, 2024.

[6] Z. Liu, C. Zhao, F. Iandola, C. Lai, Y. Tian, I. Fedorov, Y. Xiong, E. Chang, Y. Shi, R. Krishnamoorthi, L. Lai, and V. Chandra, "MobileLLM: Optimizing Sub-billion Parameter Language Models for On-Device Use Cases," in *Proc. 41st Int. Conf. Machine Learning (ICML)*, vol. 235, 2024, pp. 32431–32454.

[7] Y. Song, Z. Mi, H. Xie, and H. Chen, "PowerInfer: Fast Large Language Model Serving with a Consumer-grade GPU," in *Proc. ACM SIGOPS 30th Symp. Operating Systems Principles (SOSP '24)*, 2024, pp. 590–606, doi: 10.1145/3694715.3695964.

[8] X. Yan and Y. Ding, "Are We There Yet? A Measurement Study of Efficiency for LLM Applications on Mobile Devices," in *Proc. 2nd Int. Workshop on Foundation Models for Cyber-Physical Systems & Internet of Things (FMSys '25)*, 2025, pp. 19–24, doi: 10.1145/3722565.3727192.

[9] Y. Yang, Y. Pan, J. Zhang, and W. Ni, "Task Offloading for Collaborative Inference of LLM Agents at the Network Edge," in *Proc. IEEE 22nd Int. Conf. Mobile Ad-Hoc and Smart Systems (MASS)*, Chicago, IL, USA, Oct. 2025, pp. 525–530, doi: 10.1109/MASS66014.2025.00085.

[10] H. Li, X. Li, C. Sun, X. Wang, and V. C. M. Leung, "Edge-Enhanced Intelligence: A Comprehensive Survey of Large Language Models and Edge-Cloud Computing Synergy," *IEEE Communications Surveys & Tutorials*, vol. 28, pp. 1248–1284, 2026, doi: 10.1109/COMST.2025.3587225.

[11] M. Zhang, X. Shen, J. Cao, Z. Cui, and S. Jiang, "EdgeShard: Efficient LLM Inference via Collaborative Edge Computing," *IEEE Internet of Things Journal*, vol. 12, no. 10, pp. 13119–13131, May 2025, doi: 10.1109/JIOT.2024.3524255.

[12] S. Habibi and Ö. Erçetin, "Edge-LLM Inference With Cost-Aware Layer Allocation and Adaptive Scheduling," *IEEE Access*, vol. 13, pp. 131614–131637, 2025, doi: 10.1109/ACCESS.2025.3592308.

[13] A. Khoshsirat, G. Perin, and M. Rossi, "Decentralized LLM Inference over Edge Networks with Energy Harvesting," in *Proc. IEEE Global Communications Conf. (GLOBECOM)*, Cape Town, South Africa, Dec. 2024, pp. 3703–3708, doi: 10.1109/GLOBECOM52923.2024.10901542.

[14] Y. Chen, Y. Yan, S. Ge, Y. Qin, Y. Zheng, Q. Yang, S. He, Z. Shi, J. Chen, and Y. Shu, "Confidant: Customizing Transformer-based LLMs via Collaborative Training on Mobile Devices," in *Proc. 31st Annual Int.*

Conf. Mobile Computing and Networking (MobiCom), 2025, pp. 483–497, doi: 10.1145/3680207.3723487.

[15] I. Yaqoob, E. Ahmed, A. Gani, S. Mokhtar, M. Imran, and S. Guizani, "Mobile ad hoc cloud: A survey," *Wireless Communications and Mobile Computing*, vol. 16, no. 16, pp. 2572–2589, 2016, doi: 10.1002/wcm.2709.

[16] K. Habak, M. H. Ammar, K. A. Harras, and E. W. Zegura, "Femto Clouds: Leveraging Mobile Devices to Provide Cloud Service at the Edge," in *Proc. IEEE 8th Int. Conf. Cloud Computing (CLOUD)*, 2015, pp. 9–16, doi: 10.1109/CLOUD.2015.12.

[17] A. Mtibaa, A. Fahim, K. A. Harras, and M. H. Ammar, "Towards Resource Sharing in Mobile Device Clouds: Power Balancing Across Mobile Devices," *ACM SIGCOMM Computer Communication Review*, vol. 43, no. 4, pp. 51–56, 2013, doi: 10.1145/2491266.2491276.

[18] W. Jiang, D. Feng, Y. Sun, G. Feng, Z. Wang, and X.-G. Xia, "Joint Computation Offloading and Resource Allocation for D2D-Assisted Mobile Edge Computing," *IEEE Transactions on Services Computing*, vol. 16, no. 3, pp. 1949–1963, 2023, doi: 10.1109/TSC.2022.3190276.

[19] X. Chen, L. Jiao, W. Li, and X. Fu, "Efficient Multi-User Computation Offloading for Mobile-Edge Cloud Computing," *IEEE/ACM Transactions on Networking*, vol. 24, no. 5, pp. 2795–2808, 2016, doi: 10.1109/TNET.2015.2487344.

[20] Y. Mao, J. Zhang, and K. B. Letaief, "Dynamic Computation Offloading for Mobile-Edge Computing With Energy Harvesting Devices," *IEEE Journal on Selected Areas in Communications*, vol. 34, no. 12, pp. 3590–3605, Dec. 2016, doi: 10.1109/JSAC.2016.2611964.

[21] W. Zhang, Y. Wen, and D. O. Wu, "Energy-Efficient Scheduling Policy for Collaborative Execution in Mobile Cloud Computing," in *Proc. IEEE INFOCOM*, 2013, pp. 190–194, doi: 10.1109/INFCOM.2013.6566761.

[22] Y. Kang, J. Hauswald, C. Gao, A. Rovinski, T. Mudge, J. Mars, and L. Tang, "Neurosurgeon: Collaborative Intelligence Between the Cloud and Mobile Edge," in *Proc. 22nd Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2017, pp. 615–629, doi: 10.1145/3037697.3037698.

[23] K. Alizadeh, S. I. Mirzadeh, D. Belenko, S. Khatamifard, M. Cho, C. C. Del Mundo, M. Rastegari, and M. Farajtabar, "LLM in a Flash: Efficient Large Language Model Inference With Limited Memory," in *Proc. 62nd Annual Meeting of the Association for Computational Linguistics (ACL), Volume 1: Long Papers*, Bangkok, Thailand, Aug. 2024, pp. 12562–12584, doi: 10.18653/v1/2024.acl-long.678.

[24] F. Tan, R. Lee, Ł. Dudziak, S. X. Hu, S. Bhattacharya, T. Hospedales, G. Tzimiropoulos, and B. Martinez, "MobileQuant: Mobile-Friendly Quantization for On-Device Language Models," in *Findings of the Association for Computational Linguistics: EMNLP 2024*, Miami, FL, USA, Nov. 2024, pp. 9761–9771, doi: 10.18653/v1/2024.findings-emnlp.570.

[25] X. Lu, H. Gao, R. Wu, S. Ren, X. Chen, H. Li, and F. Li, "SmartBench: Is Your LLM Truly a Good Chinese Smartphone Assistant?," in *Proc. 2025 Conf. Empirical Methods in Natural Language Processing (EMNLP)*, Suzhou, China, Nov. 2025, pp. 3906–3931, doi: 10.18653/v1/2025.emnlp-main.194.

monic_paper_IEEE (1) (2)

ORIGINALITY REPORT

3%	2%	2%	2%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	Submitted to Panipat Institute of Engineering & Technology Student Paper	2%
2	arxiv.org Internet Source	<1%
3	www.medrxiv.org Internet Source	<1%

Exclude quotes	Off	Exclude matches	Off
Exclude bibliography	On		